DQN 實作對比報告：PyTorch vs. Keras

分析 `DQN.ipynb` 與 `DQN_keras.ipynb` 的架構差異

1. 核心框架與模型定義比較

比較項目	PyTorch (DQN.ipynb)	Keras/TF (DQN_keras.ipynb)
定義風格	命令式 (Imperative/Eager)	宣告式與靜態圖優化 (Graph Mode)
神經網路層	`torch.nn.Linear`	`layers.Dense`
資料型態轉換	需手動 `torch.from_numpy()`	內部自動處理或 `tf.constant()`

2. 梯度計算與優化機制

2.1 PyTorch 流程

PyTorch 使用 `loss.backward()` 觸發反向傳播，並透過優化器手動更新權重：

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

2.2 Keras 優化流程

Keras 版本採用了底層的 `tf.GradientTape` 以紀錄運算，並結合了 `@tf.function` 進行 JIT 編譯以加速：

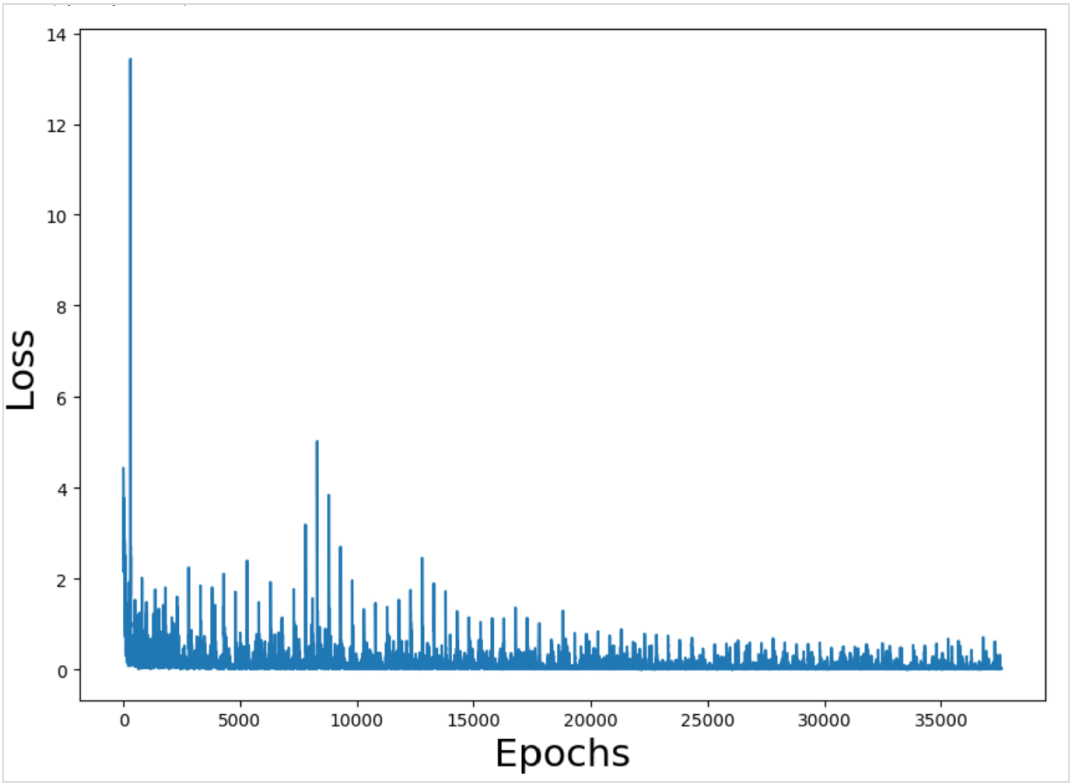
```
with tf.GradientTape() as tape:
    # 前向傳播
gradients = tape.gradient(loss, model.trainable_variables)
optimizer.apply_gradients(zip(gradients, model.trainable_variables))
```

3. 訓練效率優化 (Keras 版特色)

在 `DQN_keras.ipynb` 中，針對強化學習小批次更新的特性做了三項關鍵優化：

- **推論加速**：使用 `model(x, training=False)` 直接呼叫，避開 `predict()` 函數的高開銷。
- **編譯加速**：使用 `@tf.function` 將訓練步驟轉為靜態圖。
- **目標網路更新**：同樣具備目標網路同步機制，確保訓練穩定性。

4. 實驗結果對照



圖：DQN 訓練 Loss 曲線 (來自原始實作)

兩份程式碼雖然框架不同，但最終均能使智能體在 Gridworld 環境中達成約 90% 的勝率。PyTorch 版語法直觀，適合教學與除錯；Keras 版則展示了如何透過底層優化達到極高的訓練速度。